

7강. DevOps 개념 및 실습



한국공학대학교
TECH UNIVERSITY OF KOREA

이영곤



DevOps 등장 배경

✓ 기존 개발 방식

- 개발팀 → 코드 작성
- 운영팀 → 서비스 운영

✓ 문제점

- 배포 속도 느림
- 개발 / 운영 책임 분리
- 장애 대응 지연
- 환경 불일치

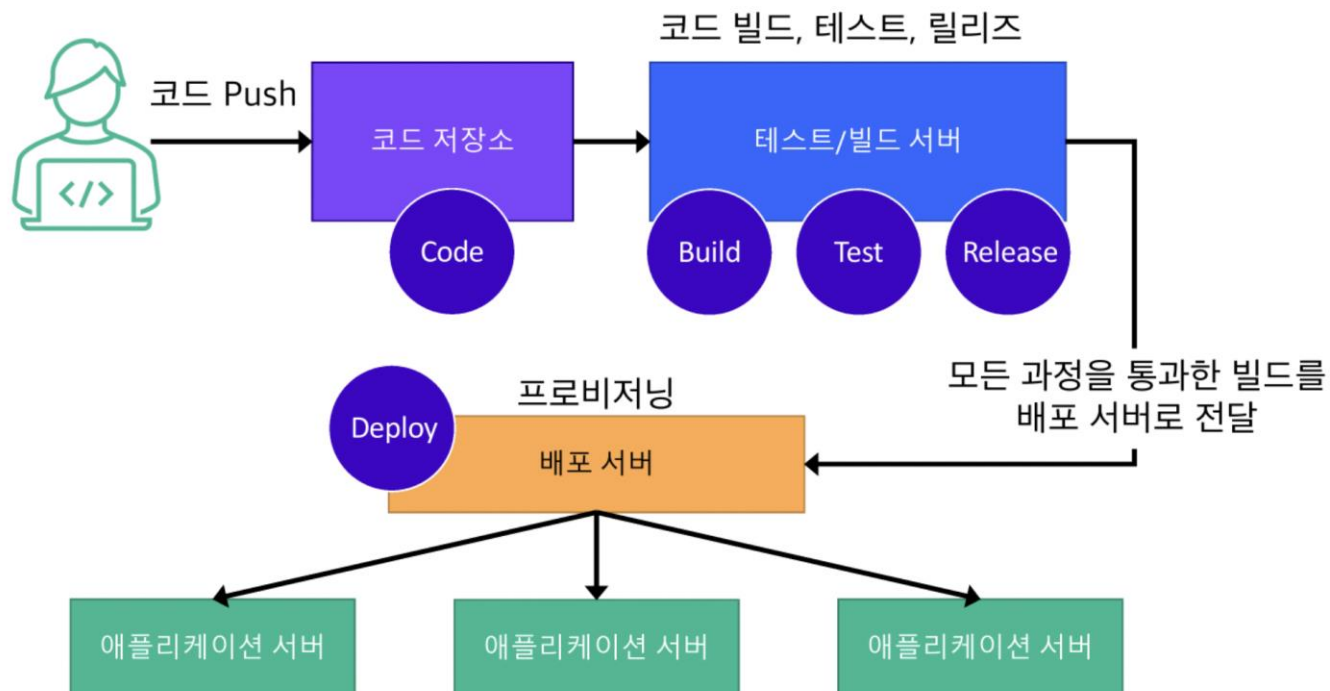


✓ DevOps 목표

- 개발 + 운영 통합
- 자동화 기반 배포
- 빠른 서비스 제공
- 안정적 운영
- 보안 유지

DevOps 구성 요소

- ✓ Continuous Integration (CI)
- ✓ Continuous Delivery (CD)
- ✓ Continuous Monitoring
- ✓ Automation
- ✓ Collaboration



1. 자동화의 무분별한 확대 금지

- ✓ CI/CD, IaC, 배포 자동화는 필수
- ✓ 검증 없는 자동화는 장애를 대규모로 확산시킴
- ✓ 아래 사항들은 반드시 갖추어야 함
 - 테스트 자동화
 - 승인 프로세스
 - 롤백 전략
 - ✓ Blue/Green 배포
 - ✓ Canary 배포

2. 보안(DevSecOps) 내재화

- ✓ 비밀키, DB 비밀번호, 인증서 하드코딩 금지
- ✓ 아래 사항 준수
 - Docker Secret / Kubernetes Secret
 - Vault
 - 이미지 취약점 스캔
 - SAST/DAST
 - RBAC 최소 권한 원칙

- ❖ SAST (Static Application Security Testing): 소스코드, 바이너리, 또는 빌드 결과물을 실행하지 않고 정적 분석하여 보안 취약점을 찾는 방식. ex. SonarQube
- ❖ DAST (Dynamic Application Security Testing): 실행 중인 애플리케이션(Web/API)을 외부 공격자 관점에서 테스트하는 방식. ex. Burp Suit

3. 모니터링 없는 배포 금지

- ✓ “배포 성공” ≠ “서비스 정상”
- ✓ 필수 요소:
 - 로그 수집 (ELK, Loki)
 - 메트릭 (Prometheus)
 - 트레이싱 (Jaeger)
 - 알림 (Alertmanager)

4. 환경 일관성 유지

- ✓ 개발/테스트/운영 환경 차이 최소화
- ✓ Docker/Kubernetes/IaC(Terraform) 적극 활용

5. 데이터 손실 방지

- ✓ DB 변경 시 가장 위험
- ✓ 아래 사항 준수
 - 백업스키마 버전 관리
 - 마이그레이션 검증
 - PV/PVC 전략
 - 복구 리허설

6. 서비스 간 의존성 관리

- ✓ API 버전 관리
- ✓ Circuit Breaker
- ✓ Retry 정책
- ✓ Timeout
- ✓ 메시지 중복 처리(Idempotency)

7. 배포 속도보다 안정성 우선

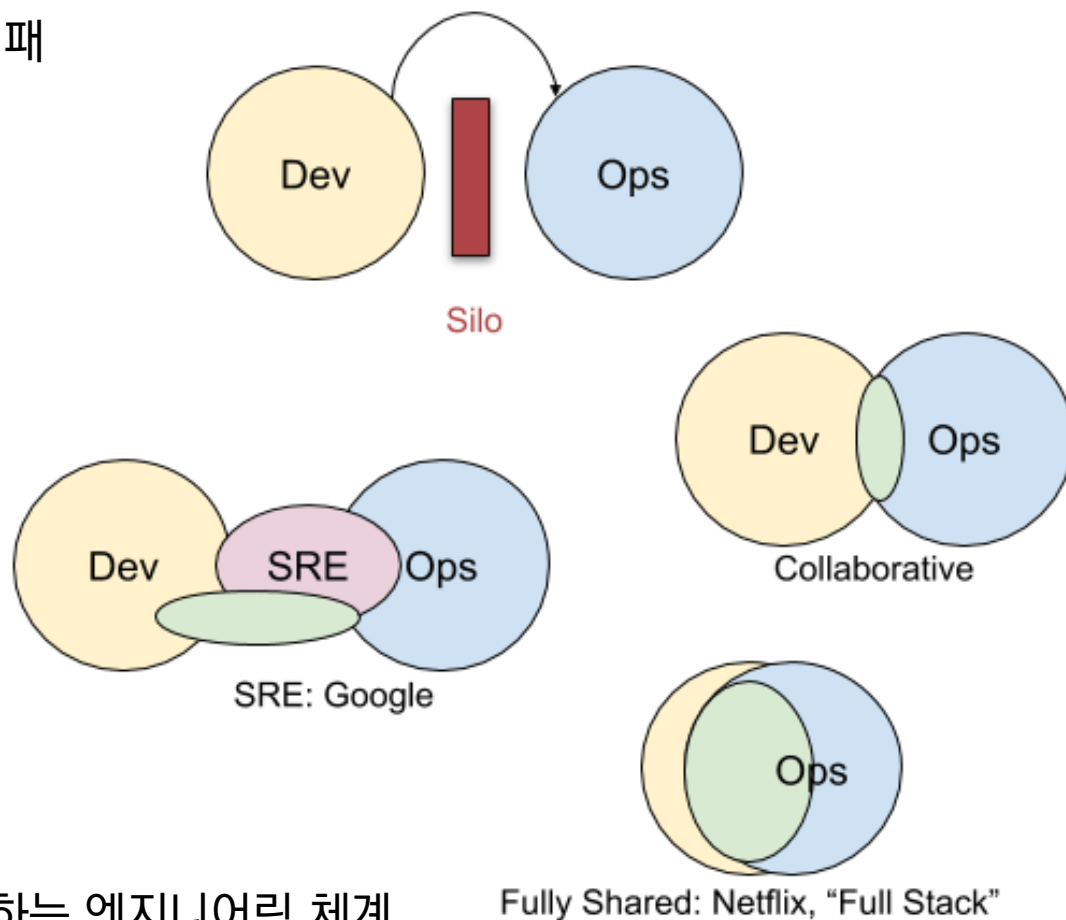
- ✓ 잦은 배포 자체가 목표가 아님
- ✓ KPI
 - MTTR
 - Deployment Frequency
 - Change Failure Rate

- ❖ MTTR (Mean Time To Recovery)
- ❖ Change Failure Rate: 배포 변경이 장애/롤백/핫픽스를 유발한 비율

DORA Metrics(DevOps Research and Assessment)

8. 조직 문화 문제

- ✓ Dev vs Ops 분리 문화 지속 시 실패
- ✓ 핵심 사항:
 - 공동 책임
 - 협업
 - 표준화
 - 문서화



❖ SRE(Site Reliability Engineering)

- DevOps를 운영 안정성 중심으로 구현하는 엔지니어링 체계
- 시스템 운영, 가용성, 확장성, 복구성을 관리하는 역할
- 운영(Ops)을 수작업이 아닌 코드와 자동화로 해결

9. 비용 관리

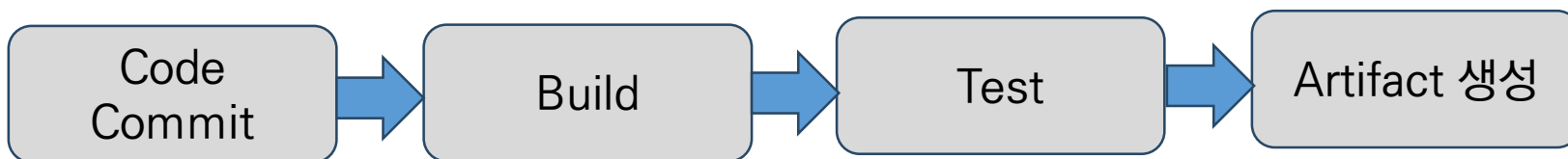
- ✓ 클라우드/K8s는 방치 시 비용 폭증
- ✓ 주의 사항
 - 과도한 리소스 요청
 - 미사용 Pod
 - 로그 과다 저장
 - GPU/스토리지 낭비

10. 재해 복구 계획

- ✓ 장애는 반드시 발생
- ✓ 대응:
 - DR 시나리오, 다중 AZ, 백업, 복원 테스트, 운영 매뉴얼

Continuous Integration (CI)

- ✓ 개발자가 작성한 코드를 자동으로 빌드 및 테스트



- ✓ 대표 도구: Jenkins, GitHub Actions, GitLab CI, CircleCI

Continuous Delivery / Deployment

✓ Continuous Delivery

- 자동 빌드 + 테스트
- 배포는 수동 승인

✓ Continuous Deployment

- 자동 빌드
- 자동 테스트
- 자동 배포



GitOps 정의

- ✓ Git을 단일 진실 원천(Single Source of Truth) 으로 사용하여 인프라 및 애플리케이션을 관리하는 운영 방식
 - Git = 시스템 상태 = 현재 운영 상태의 기준 문서

Git 변경이 배포 트리거

- Git commit
- Pull Request
- Merge

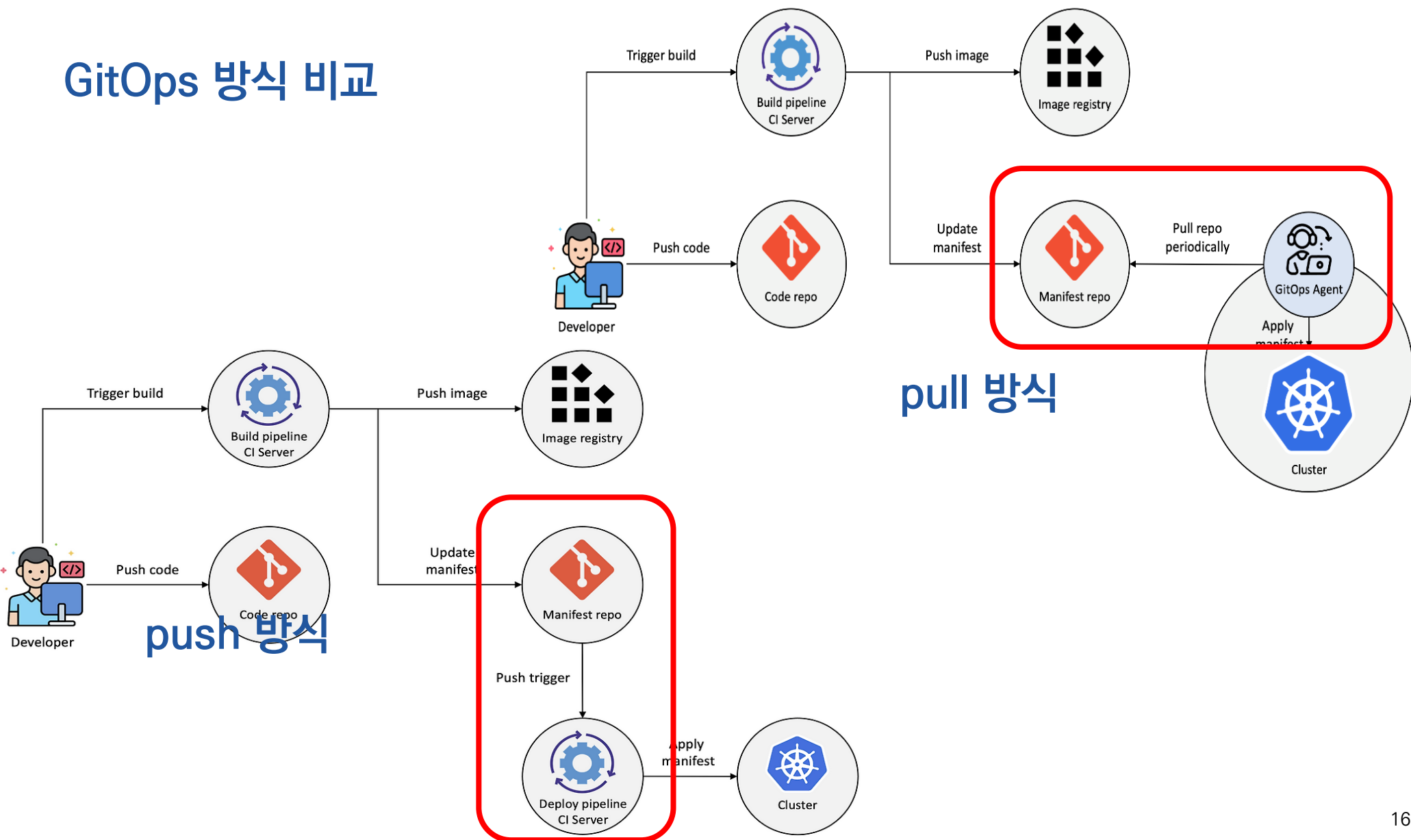
GitOps 핵심 원칙

- ✓ 선언적 구성
 - Infrastructure as Code
 - Kubernetes YAML
 - Helm
- ✓ Git 단일 소스
 - 모든 상태는 Git 저장
 - deployment.yaml
 - service.yaml
- ✓ 자동 동기화
 - Git 변경 → 시스템 반영
- ✓ Pull 방식
 - 시스템이 Git을 감시
 - Cluster → Git

GitOps 핵심 도구

방식	선택/필수	역할	비고
Kubernetes YAML	선택	가장 기본적인 배포 정의	
Helm Chart	선택	템플릿 기반 배포	복잡한 서비스 패키징. 변수 관리. 버전 관리 적합
Kustomize	선택	dev/stage/prod 환경별 오버레이	YAML 원본 유지. 가독성 좋음. 단순 환경 분리 명확
Terraform	선택	클라우드/인프라 자원 생성	IaC 실행시 활용
Argo CD / Flux	필수	Git 상태와 클러스터 상태 동기화	Flux는 UI보다 선언형 자동화와 이미지 업데이트 자동화를 더 강하게 요구시 활용

GitOps 방식 비교



GitOps 구성 요소

- ✓ Git Repository
- ✓ CI Pipeline
- ✓ Container Registry
- ✓ GitOps Controller
- ✓ Kubernetes Cluster

Git Repository

- ✓ 프로젝트 파일과 모든 변경 기록을 저장하는 구조
- ✓ 단순 파일 폴더 + 변경 이력 DB

기능	설명
Version Control	변경 이력 저장
Branching	기능별 분기
Merge	코드 통합
Rollback	이전 상태 복구
Collaboration	팀 협업
Audit	누가 무엇을 바꿨는지 추적

Git Repository

항목	GitHub	GitLab
SaaS	업계 최고 수준 SaaS 플랫폼	성숙한 SaaS 플랫폼
온프레미스	Enterprise 별도 구축 필요	완전한 Self-host 지원
CI/CD	GitHub Actions 기반 확장형	기본 내장형 통합 플랫폼
DevSecOps 통합	외부 도구 연계 중심	보안 기능 기본 통합
UI/UX	사용자 친화적	기능 중심
오픈소스 생태계	최대 규모 생태계	강력한 자체 생태계
기업 보안 통제	우수	엔터프라이즈급 통제
비용	SaaS 중심, Enterprise 비용 높음	Self-host 기반 비용 효율

CI Pipeline

- ✓ 개발자가 코드 변경을 Git에 Push할 때마다 자동으로 빌드, 테스트, 품질검사, 보안검사 등을 수행하는 자동화된 검증 프로세스
- ✓ 코드 변경을 빠르고 안전하게 통합
 - 코드 충돌 조기 발견
 - 품질 유지
 - 배포 준비 상태 확보
 - 자동화된 신뢰성 강화

Git Push
→ GitHub Actions
→ Maven Build
→ JUnit
→ SonarQube
→ Docker Build
→ Harbor Push

Container Registry

- ✓ Docker/OCI 컨테이너 이미지를 저장, 버전 관리, 배포하기 위한 중앙 저장소
- ✓ 컨테이너 빌드 후 컨테이너 이미지는 로컬 PC에만 존재
- ✓ 운영 환경 배포하려면 아래 환경에서 사용가능해야 함
 - Kubernetes
 - 다른 서버
 - CI/CD 개발팀

모두가 접근 가능한 저장소 필요

⇒ Container Registry 사용

Container Registry 기능

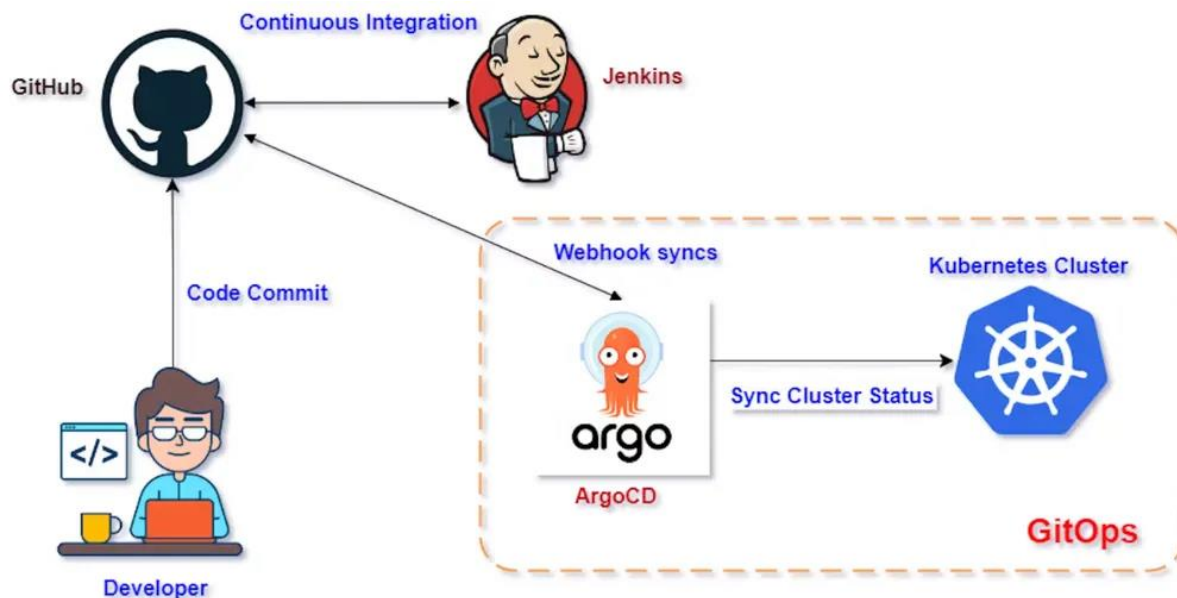
기능	설명
Image Storage	이미지 저장
Version Tagging	v1.0.1 등
Access Control	사용자/팀 권한
Vulnerability Scan	보안 취약점 검사
Replication	다중 환경 복제
Signature	이미지 서명
Audit	누가 배포했는지 추적

Container Registry 비교

항목	Docker Hub	Harbor
운영 형태	SaaS 기반	Self-host / Private Registry
공개 이미지	글로벌 최대 규모	제한적
프라이빗 레지스트리	지원	엔터프라이즈급
온프레미스	Enterprise 중심	완전 지원
보안 통제	표준 수준	고도화된 정책 제어
취약점 스캔	일부 지원	기본 내장
이미지 서명	부분 지원	정책 기반 지원
RBAC	기본 권한 관리	세분화된 접근 제어
복제	제한적	멀티 사이트 지원
속도	인터넷 환경 의존	내부망 최적화
비용 통제	구독 기반	자체 운영 최적화
기업 DevOps 적합성	중견 수준	대기업/온프레미스 최적

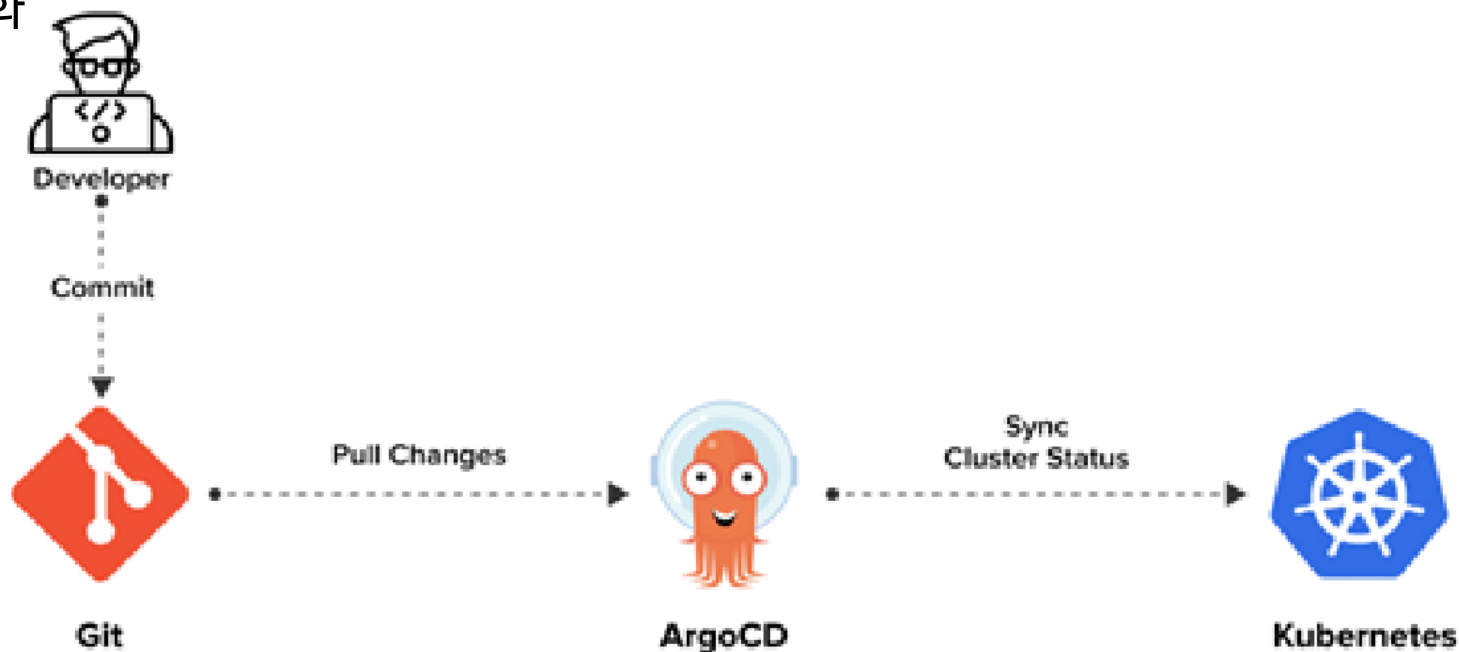
GitOps Controller

- ✓ Git 저장소에 정의된 원하는 시스템 상태(desired state)와 실제 Kubernetes/인프라 상태를 지속적으로 비교하고 자동으로 일치시키는 운영 자동화 엔진
 - 실제 클러스터 상태 $\neq$ Git 상태 $\rightarrow$ 자동 수정(Reconciliation)
- ✓ GitOps Controller = “자동 배포 + 지속적 상태 감시 + Drift 복구 시스템”
- ✓ 대표 도구: Argo CD, Flux



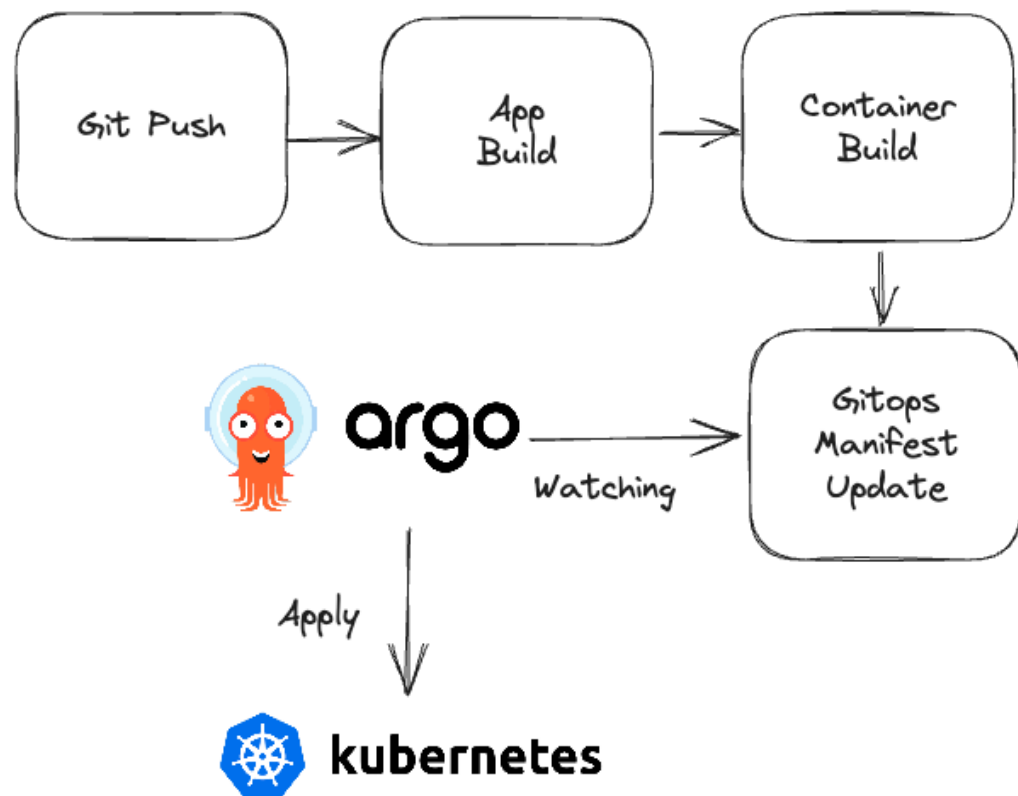
ArgoCD: Kubernetes용 GitOps Deployment Tool

- ✓ Kubernetes 배포 자동화
- ✓ Git 상태 감시
- ✓ 자동 동기화
- ✓ 롤백 지원

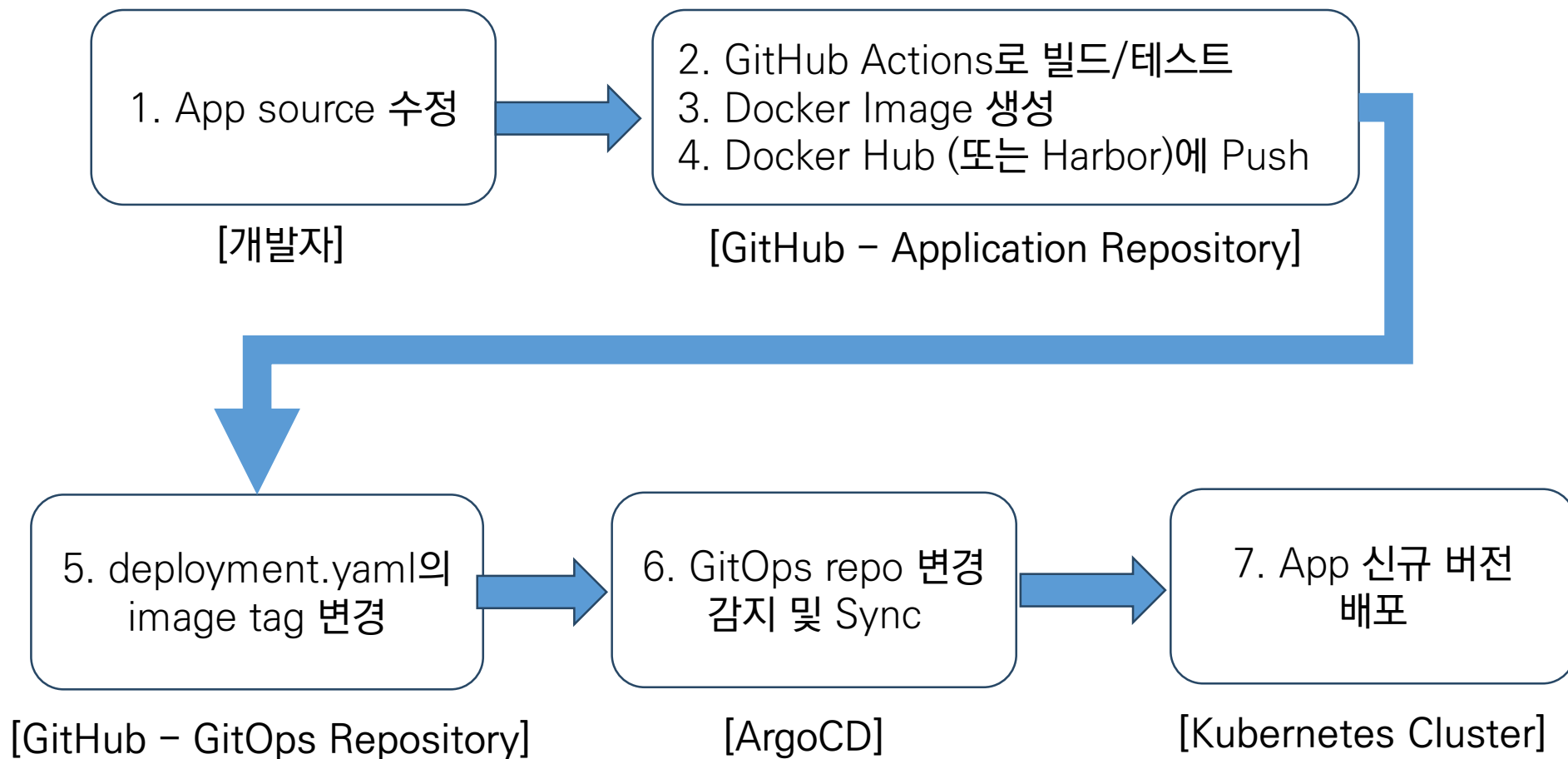


ArgoCD 동작 과정

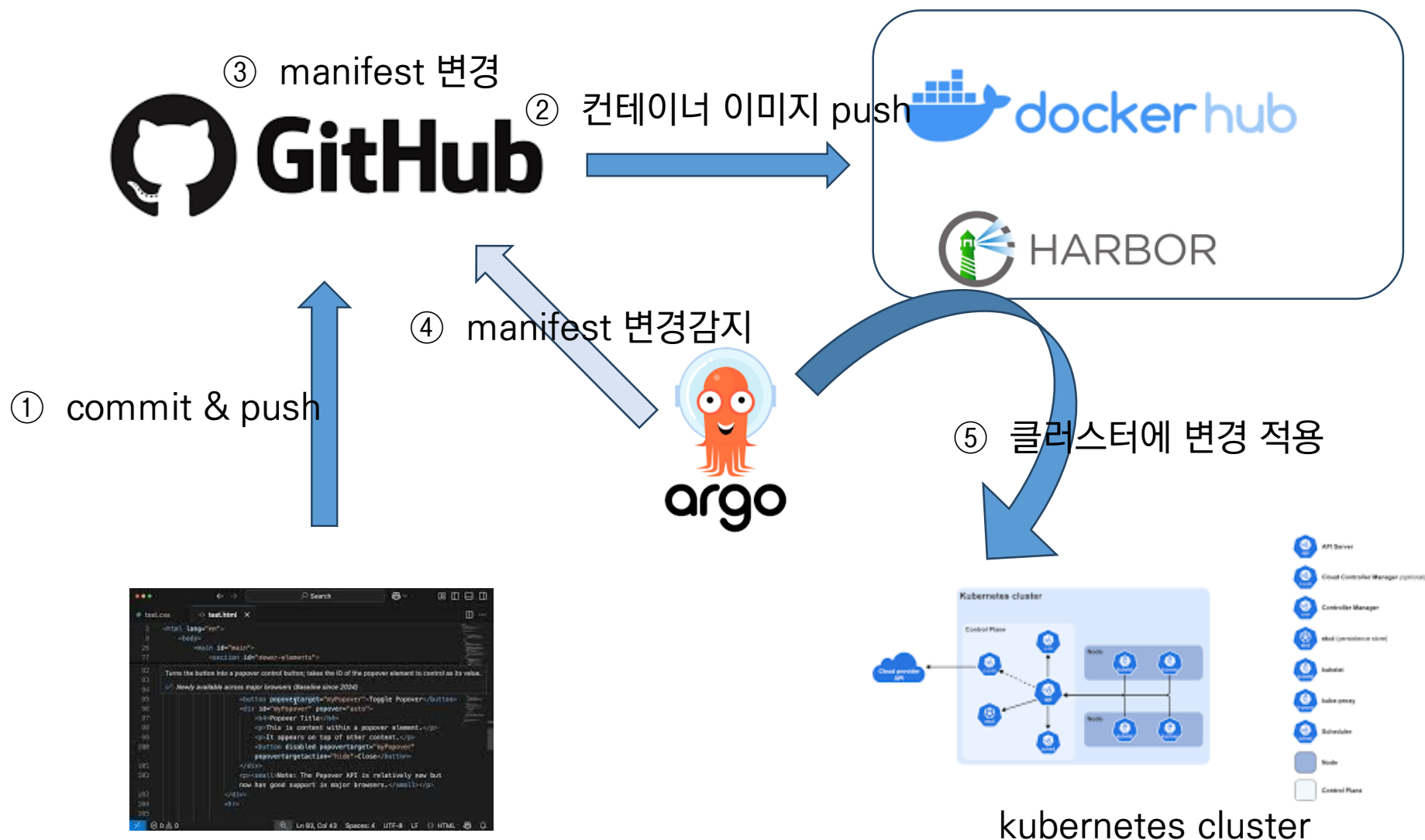
- ✓ Git Repository 확인
 - deployment.yaml
 - service.yaml
- ✓ Cluster 상태 비교
 - Desired State
 - Actual State
- ✓ 차이 발생
 - Sync 수행
- ✓ Kubernetes 배포



실습 과정



실습 과정



애플리케이션 레포지토리

book-api

- src
 - main
 - java/com/example/bookapi
 - BookApiApplication.java
 - controller/BookController.java
 - model/Book.java
 - service/BookService.java
 - resources
 - application.yml
- pom.xml
- Dockerfile
- .github
 - workflows
 - ci.yaml

구성

역할

- ✓ Java 코드 관리
- ✓ Maven 빌드테스트
- ✓ Docker 이미지 생성
- ✓ Registry Push

GitOps 레포지토리

구성

```
book-api-gitops
├── base
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
├── overlays
│   └── dev
│       ├── namespace.yaml
│       └── kustomization.yaml
```

역할

- ✓ Kubernetes manifest 관리
- ✓ 배포 버전(image tag) 관리
- ✓ 환경(dev/prod)별 설정 관리
- ✓ ArgoCD가 참조하는 기준 저장소

GitOps 레포지토리와 App 레포지토리 동일 혹은 분리시 CI/CD

동일

- ① Git Push
- ② Build
- ③ Docker Hub Push
- ④ k8s/deployment.yaml 이미지 태그 수정
- ⑤ 같은 repo commit
- ⑥ ArgoCD Sync

분리

- ① App Repo Push
- ② Build
- ③ Docker Hub Push
- ④ GitOps Repo deployment.yaml 수정
- ⑤ GitOps Repo commit
- ⑥ ArgoCD Sync

CI.yml

Docker hub에 있는 CI
서버 사용

```
name: CI/CD Pipeline
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
env:
```

```
  DOCKERHUB_USERNAME:
```

```
  ${{ secrets.DOCKERHUB_USERNAME }}
```

```
  IMAGE_NAME: resttemplateserver-hub
```

```
jobs:
```

```
  build-and-push:
```

```
    name: Build & Push to Docker Hub
```

```
    runs-on: ubuntu-latest
```

```
  outputs:
```

```
    image_tag: ${{ steps.meta.outputs.tag }}
```

```
steps:
```

```
  - name: Checkout source
```

```
    uses: actions/checkout@v4
```


CI.yml

- name: Set up JDK 17
uses: actions/setup-java@v4
with:
 java-version: '17'
 distribution: 'temurin'
 cache: maven
- name: Grant execute permission for Maven Wrapper
run: chmod +x ./mvnw
- name: Build with Maven
run: ./mvnw clean package -DskipTests
- name: Generate image tag
id: meta
run: echo "tag=\${{ github.sha }}" >> \$GITHUB_OUTPUT
- name: Log in to Docker Hub
uses: docker/login-action@v3
with:
 username: \${{ secrets.DOCKERHUB_USERNAME }}
 password: \${{ secrets.DOCKERHUB_TOKEN }}

CI.yml

```
- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: |
      ${ env.DOCKERHUB_USERNAME }/${ env.IMAGE_NAME }:
      ${ env.DOCKERHUB_USERNAME }/${ env.IMAGE_NAME }:latest

update-manifest:
  name: Update K8s Manifest for ArgoCD
  runs-on: ubuntu-latest
  needs: build-and-push
```

CI.yml

steps:

- name: Checkout source
uses: actions/checkout@v4
with:
token: \${ secrets.GITHUB_TOKEN }
- name: Update image tag in k8s.yaml
run: |
NEW_IMAGE="\${ env.DOCKERHUB_USERNAME }/\${ env.IMAGE_NAME }:---"
sed -i "s|image: .*\${ env.IMAGE_NAME }.*|image: \${NEW_IMAGE}|g" k8s.yaml
- name: Commit and push manifest update
run: |
git config user.name "github-actions[bot]"
git config user.email "github-actions[bot]@users.noreply.github.com"
git add k8s.yaml
git diff --staged --quiet || git commit -m "ci: update image to ---}"
git push

k8s.yaml의 이미지태그
교체

manifest 파일 수정 커밋

CI.yml에 의한 이미지 태그 교체 (SHA:Secure Hash Algorithm 태그) 이유

- ✓ latest만 사용할 경우, latest = mutable
 - 같은 태그로 내용 변경 가능 -> 추적 어려움
 - 재현성 낮음
 - 복구 어려움

방식	유일성	롤백	추적성
latest	낮음	어려움	낮음
SHA	높음	쉬움	높음

역할 구분

CI 역할

- ✓ Application repo에서:소스 checkout
- ✓ Maven build/test
- ✓ Docker image build
- ✓ Docker registry push
- ✓ GitOps repo checkout
- ✓ 새 image tag 반영
- ✓ GitOps repo commit/push

ArgoCD 역할

- ✓ GitOps repo 변경 감시
- ✓ desired state와 actual state 비교
- ✓ 차이 감지 시 sync
- ✓ Kubernetes 배포 반영

ArgoCD.yml

툴에 의해
자동생성

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: book-api-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-id/book-api-gitops.git
    targetRevision: HEAD
    path: overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: book-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Preparation at Docker hub ⇒ generate token

The screenshot shows the Docker Hub account settings page for user 'yklee2002'. The left sidebar contains navigation links: Home, Hub, Build Cloud, Hardened Images, Scout, Testcontainers Cloud, Docker Desktop, Settings, Account information, Email, Password, 2FA, Personal access tokens (highlighted with a red circle), Connected accounts, Convert, and Privacy. The main content area is titled 'Personal access tokens' and includes a description: 'You can use a personal access token instead of a password for Docker CLI authentication. Create multiple tokens, control their scope, and delete tokens at any time. [Learn more](#)'. Below this is a table of tokens.

Description	Scope	Status	Source	Created	Last used	
my token	Read, Write, Delete	Active	Manual	Mar 12, 2026 at 12:16:16	Mar 12, 2026 at 12:19:02	
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Feb 23, 2026 at 12:56:47	Mar 12, 2026 at 11:38:35	
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Jun 09, 2025 at 10:55:02	Mar 09, 2026 at 12:39:53	Never
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Feb 21, 2026 at 18:21:56	Feb 23, 2026 at 12:30:41	Never
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Jan 01, 2026 at 15:55:25	Feb 21, 2026 at 14:18:08	Never
my token3	Read, Write, Delete	Active	Manual	Dec 03, 2025 at 12:02:44	Feb 20, 2026 at 23:32:40	Expired

On the right, a user profile dropdown menu is open, showing options: What's new, My profile, Account settings (circled in red), Billing, and Sign out.

Preparation at Docker hub ⇒ generate token

The screenshot displays the Docker Hub interface for creating a personal access token. The left sidebar shows the user profile 'ykle2002' and navigation options. The 'Personal access tokens' option is highlighted and circled in red. The main content area shows the 'Create access token' form with the following fields:

- Access token description:** A text input field.
- Expiration date:** A dropdown menu set to 'None'.
- Access permissions:** A dropdown menu set to 'Read, Write, Delete', which is circled in red.

Below the form, there are two buttons: 'Cancel' and 'Generate'.

Preparation at Github: set secret for github access

leeyoungkon / book-api

Code Issues Pull requests Actions Projects Wiki Security Insights **Settings**

General

Access

- Collaborators
- Moderation options

Code and automation

- Branches
- Tags
- Rules
- Actions
- Models Preview
- Webhooks
- Copilot
- Environments
- Codespaces
- Pages

Security

- Advanced Security
- Deploy keys
- Secrets and variables**

General

Repository name

book-api Rename

☐ **Template repository**

Template repositories let users generate new repositories with the same directory structure and files. [Learn more about template repositories.](#)

Default branch

The default branch is considered the “base” branch in your repository, against which all pull requests and code commits are automatically made, unless you specify a different branch.

main

Releases

☐ **Enable release immutability**

Disallow assets and tags from being modified once a release is published.

Social preview

Upload an image to customize your repository’s social media preview.

Images should be at least 640×320px (1280×640px for best display).

[Download template](#)

Preparation at Github: set secret for github access

leeyoungkon / book-api

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security

Advanced Security

Deploy keys

Secrets and variables

Actions

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets Variables

Environment secrets

This environment has no secrets.

Manage environment secrets

Repository secrets

New repository secret

Name ↕	Last updated
DOCKERHUB_TOKEN	15 minutes ago
DOCKERHUB_USERNAME	19 minutes ago

Preparation at Github: set secret for github access

leeyoungkon / book-api

Issues Pull requests Actions Projects Wiki Security Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security

Advanced Security

Deploy keys

Secrets and variables

Actions

Actions secrets / New secret

Name *

DOCKERHUB_USERNAME

Secret *

Preview

Add secret

Preparation at Github: set secret for github access

The screenshot shows the GitHub interface for the repository 'leeyoungkon / book-api'. The 'Settings' tab is selected, and the 'Actions secrets / New secret' page is displayed. The 'Name' field is highlighted with a red circle and contains the text 'DOCKERHUB_TOKEN'. The 'Secret' field is empty. A green 'Add secret' button is visible at the bottom right of the form. The left sidebar shows the repository's navigation menu, with 'Secrets and variables' expanded under the 'Security' section.

leeyoungkon / book-api

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security

Advanced Security

Deploy keys

Secrets and variables

Actions secrets / New secret

Name *

DOCKERHUB_TOKEN

Secret *

Preview

Add secret

Preparation at Github: workflow 실행 위치 결정

- ✓ GitHub Hosted Runner
 - GitHub가 제공하는 서버에서 실행
 - runs-on: ubuntu-latest
 - 흐름: GitHub Push → GitHub 서버에서 Runner 실행 → Build/Test/Push
 - 로컬 PC 필요 없음
- ✓ Self-hosted Runner
 - 내 PC 또는 사내 서버에서 실행
 - runs-on: self-hosted
 - 흐름: GitHub Push → GitHub가 내 Runner에 작업 전달 → 내 PC에서 실행
 - 로컬 Runner 필요

Preparation at Github: self-hosted runner 필요한 경우

- ✓ GitHub 서버가 직접 접근 못 하는 자원이 있는 경우:
 - 사내 Harbor
 - 내부 Kubernetes
 - 사설망 DB
 - 로컬 Docker Desktop
 - 온프레미스 인프라

Preparation at Github: self-hosted runner 등록 ⇒ cmd에서 수행

Runners / Add new self-hosted runner · leeyoungkon/resttemplateserver-hub

Add new self-hosted runner · leeyoungkon/resttemplateserver-hub

Warning: Using self-hosted runners in public repositories is not recommended. Forks of your public repository can potentially run dangerous code on your self-hosted runner by creating a pull request. [Learn more about security hardening for self-hosted runners.](#)

Adding a self-hosted runner requires that you download, configure, and execute the GitHub Actions Runner. If you do not already have an existing volume licensing agreement for your GitHub purchases, by downloading and configuring the GitHub Actions Runner, you agree to the [GitHub Customer Agreement](#).

Runner image

☐ macOS ☐ Linux ☒ Windows

Architecture

x64

Download

We recommend configuring the runner under "actions-runner". This will help avoid issues related to service identity folder permissions and long path restrictions on Windows.

```
# Create a folder under the drive root
$ mkdir actions-runner; cd actions-runner

# Download the latest runner package
$ Invoke-WebRequest -Uri https://github.com/actions/runner/releases/download/v2.334.0/actions-runner-win-x64-2.334.0.zip -OutFile actions-runner-win-x64-2.334.0.zip

# Optional: Validate the hash
$ if((Get-FileHash -Path actions-runner-win-x64-2.334.0.zip -Algorithm SHA256).Hash.ToUpper() -ne 'a0c896f3acf37841cc17f392a38111d39501e56f2990434567f027ee89cf8981'.ToUpper()){ throw 'Computed checksum did not match expected checksum' }
```

설치 후 서비스로 전환하면 편리함

- svc install
- svc start

Preparation at Github: action $\Rightarrow$ general $\Rightarrow$ W/F permission 부여

leeyoungkon / resttemplateserver-hub

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

General

Runners

OIDC

Actions permissions

☒ **Allow all actions and reusable workflows**
Any action or reusable workflow can be used, regardless of who authored it or where it is defined.

☐ **Disable actions**
The Actions tab is hidden and no workflows can run.

☐ **Allow leeyoungkon actions and reusable workflows**
Any action or reusable workflow defined in a repository within leeyoungkon can be used.

☐ **Allow leeyoungkon, and select non-leeyoungkon, actions and reusable workflows**
Any action or reusable workflow that matches the specified criteria, plus those defined in a repository within leeyoungkon. [Learn more](#)

Workflow permissions

☐ **Require**
Choose the default permissions granted to the GITHUB_TOKEN when running workflows in this repository. You can specify more granular permissions in the workflow using YAML. [Learn more about managing permissions.](#)

☒ **Read and write permissions**
Workflows have read and write permissions in the repository for all scopes.

☐ **Read repository contents and packages permissions**
Workflows have read permissions in the repository for the contents and packages scopes only.

Save

Execute CI workflow

The screenshot displays the GitHub Actions interface for a repository named 'leeyoungkon / book-api'. The 'Actions' tab is selected and highlighted with a red circle. Below the repository name, the workflow 'Enhance CI workflow for Docker image handling #17' is shown with a green checkmark. The left sidebar contains a 'Summary' section with 'All jobs' and a list of jobs: 'build' (marked with a green checkmark), 'Run details', 'Usage', and 'Workflow file'. The main content area shows the workflow details for the 'build' job, which is also highlighted with a red circle. It indicates that the job was triggered 7 minutes ago, has a status of 'Success', and a total duration of '1m 14s'. The workflow file is 'ci.yml' and it runs on 'push'. The 'build' job is shown as a green checkmark with a duration of '1m 10s'. At the bottom, the 'Annotations' section shows '1 warning'.

leeyoungkon / book-api

<> Code Issues Pull requests **Actions** Projects Wiki Security Insights Settings

← CI

✓ Enhance CI workflow for Docker image handling #17

Summary

All jobs

✓ build

Run details

Usage

Workflow file

Re-run triggered 7 minutes ago

Status

Total duration

Artifacts

leeyoungkon - fe9069 main Success 1m 14s -

ci.yml

on: push

✓ build 1m 10s

Annotations

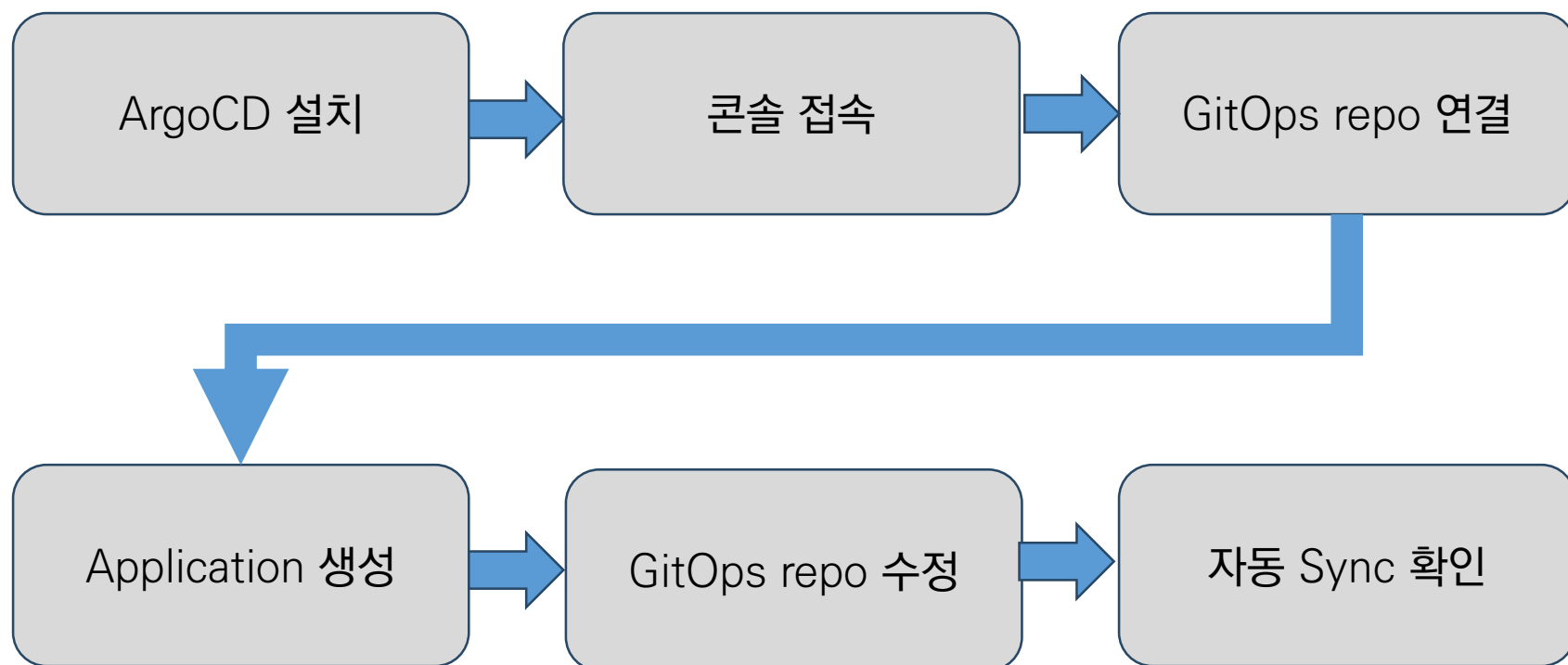
1 warning

Confirm docker image at docker hub

The screenshot shows the Docker Hub interface for the user 'yklee2002'. The left sidebar contains navigation links: Repositories, Hardened Images, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area is titled 'Repositories' and lists all repositories within the 'yklee2002' namespace. A search bar and a dropdown for 'All content' are present. A table lists the repositories with columns for Name, Last Pushed, Contains, Visibility, and Scout. The repository 'yklee2002/book-api' is circled in red.

Name	Last Pushed	Contains	Visibility	Scout
yklee2002/book-api	10 minutes ago	IMAGE	Public	Inactive
yklee2002/demo	4 days ago	IMAGE	Public	Inactive
yklee2002/msa	5 days ago	IMAGE	Public	Inactive
yklee2002/second	20 days ago	IMAGE	Public	Inactive
yklee2002/receiver	3 months ago	IMAGE	Public	Inactive
yklee2002/sender	3 months ago	IMAGE	Public	Inactive
yklee2002/user	4 months ago	IMAGE	Public	Inactive
yklee2002/maker	4 months ago	IMAGE	Public	Inactive

Process



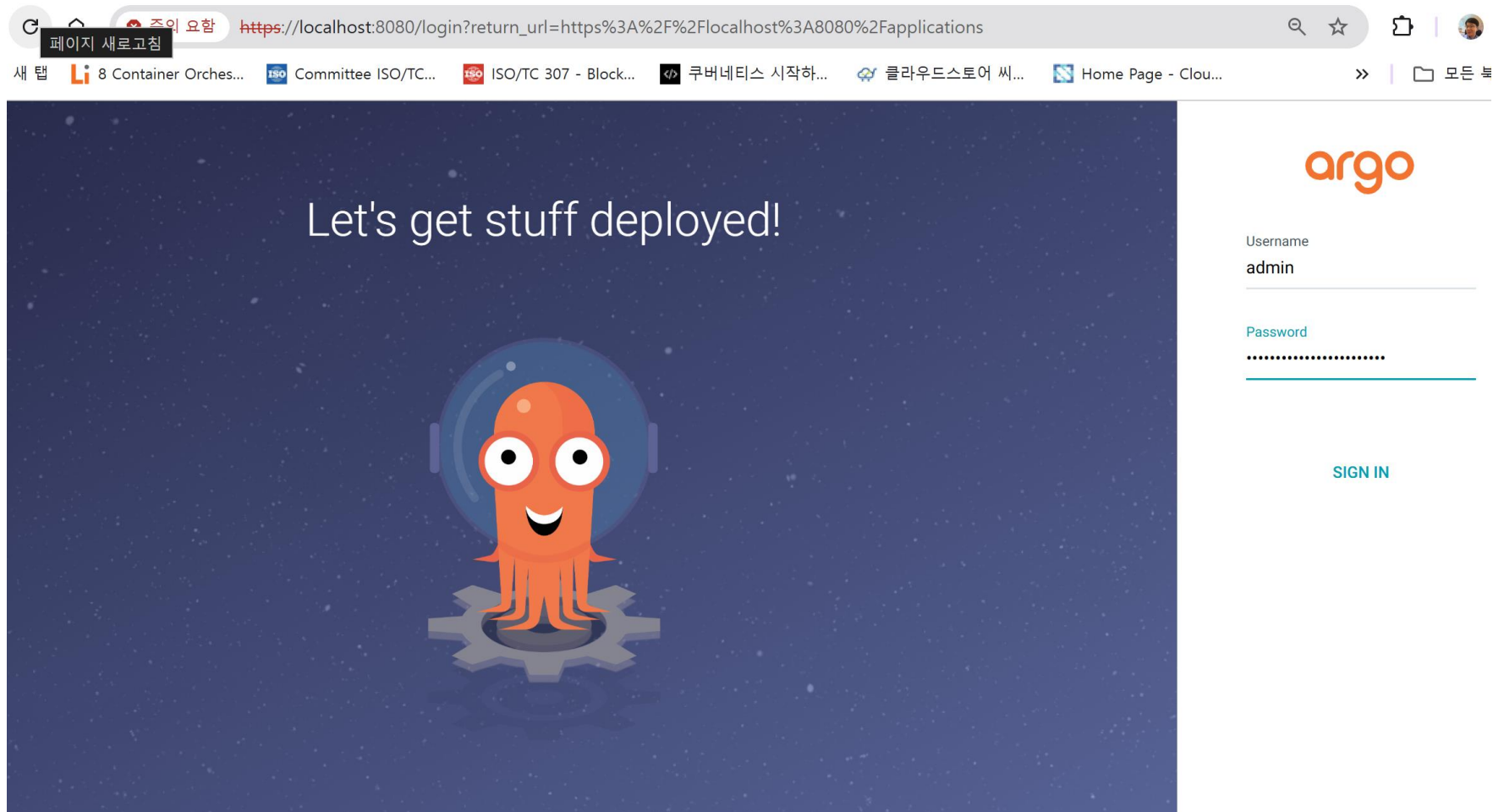
ArgoCD 설치를 위한 컨텍스트 확인

- ✓ `kubectl config current-context` //현재 컨텍스트 확인
- ✓ `kubectl get nodes` //노드 확인
- ✓ `kubectl config get-contexts` // 컨텍스트 정보 보기
- ✓ `kubectl cluster-info` //현재 사용중인 클러스터 정보 보기
- ✓ `kubectl config view` //컨피그 뷰 보기
- ✓ `kubectl config use-context default` //컨텍스트를 default로 전환

ArgoCD 설치

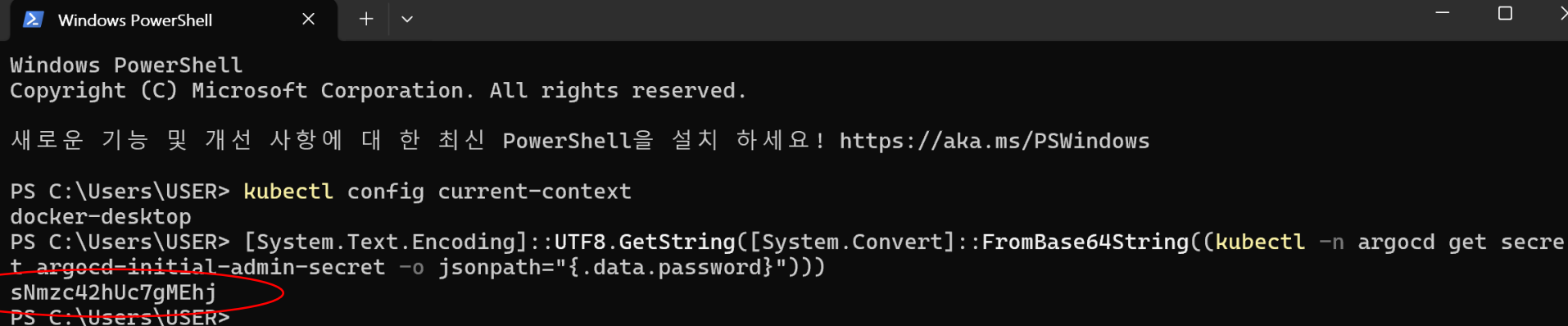
- ✓ `kubectl create namespace argocd`
- ✓ `kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml`
- ✓ `kubectl get pods -n argocd`
- ✓ `kubectl port-forward svc/argocd-server -n argocd 8080:443`
- ✓ `kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d && echo` //linux 서버에서
- ✓ `[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String((kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}"))))` //윈도우 powershell에서

ArgoCD UI 접속: localhost:8080



ArgoCD 로그인

- ✓ ID: admin
- ✓ password: 아래쪽 그림과 같이 powershell에서 생성된 비밀번호 입력



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 기능 및 개선 사항에 대한 최신 PowerShell을 설치 하세요! https://aka.ms/PSWindows

PS C:\Users\USER> kubectl config current-context
docker-desktop
PS C:\Users\USER> [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String((kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}")))
sNmzc42hUc7gMEhj
PS C:\Users\USER>
```

개발된 앱에 CD 적용 과정

The screenshot shows the ArgoCD web interface at `https://localhost:8080/applications`. The browser's address bar and tabs are visible at the top. The interface has a dark sidebar on the left with the Argo logo and version `v3.3.3`, and navigation links for Applications, Settings, User Info, and Documentation. The main content area is titled 'Applications' and features a toolbar with buttons for '+ NEW APP' (circled in red), 'SYNC APPS', and 'REFRESH APPS', along with a search bar. Below the toolbar, there is a large circular icon representing a stack of applications and a message: 'No applications available to you just yet. Create new application to start managing resources in your namespace.' A 'CREATE APPLICATION' button is located at the bottom right of the main area.

ArgoCD에서 애플리케이션 생성

- ✓ Name: book-api-dev
- ✓ Project: default
- ✓ Sync policy: manual (처음에는)
- ✓ Repository URL: <https://github.com/본인아이디/book-api.git>
- ✓ Revision: HEAD
- ✓ Path: .
- ✓ Cluster URL: <https://kubernetes.default.svc>
- ✓ Namespace: default
- ✓ 마지막에 create 버튼 누름

ArgoCD에서 애플리케이션 sync 함

The screenshot shows the ArgoCD web interface. The left sidebar contains the Argo logo (v3.3.3) and navigation links: Applications, Settings, User Info, and Documentation. Below these are application filters for Favorites Only, SYNC STATUS (Unknown, Synced, OutOfSync), and HEALTH STATUS (Progressing). The main content area displays the 'book-api-dev' application card. The card shows the following details:

- Project:** default
- Labels:**
- Status:** Healthy Synced (indicated by a green heart and checkmark)
- Repository:** https://github.com/leeyoungkon/book-api.git
- Target Revi...** HEAD
- Path:** overlays/dev
- Destination:** in-cluster
- Namespace:** book-dev
- Created At:** 03/13/2026 12:36:39 (26 minutes ago)
- Last Sync:** 03/13/2026 13:00:08 (2 minutes ago)

At the bottom of the card are three buttons: SYNC, REFRESH, and DELETE. A red circle highlights the 'sync' icon (a square with a circular arrow) in the top right corner of the application card.

ArgoCD에서 실행상태 확인

← → ↺ 🏠 주의 요함 <https://localhost:8080/applications/argocd/book-api-dev?view=tree&resource=>

📦 새 탭 8 Container Orches... 100 Committee ISO/TC... 100 ISO/TC 307 - Block... <📄> 쿠버네티스 시작하... 🌐 클라우드스토어 써... 🏠 Home Page - Clou... 📡 [시장동향] 대세는... 📡 큰 숲 맑은 샘 📡 대한성서



- Applications
- Settings
- User Info
- Documentation

Resource filters

NAME

NAME

KINDS

KINDS

SYNC STATUS

☐ Synced 3

☐ OutOfSync 0

HEALTH STATUS

☐ Progressing 0

Applications / 🔍 book-api-dev

DETAILS DIFF SYNC SYNC STATUS HISTORY AND ROLLBACK DELETE REFRESH

APP HEALTH 🟢 **Healthy**

SYNC STATUS 🟢 **Synced** to HEAD (af9cb53) [🔗](#)

Auto sync is not enabled.

Author: leeyoungkon <yklee2002@gmail.com> -

Comment: libe probe 지움

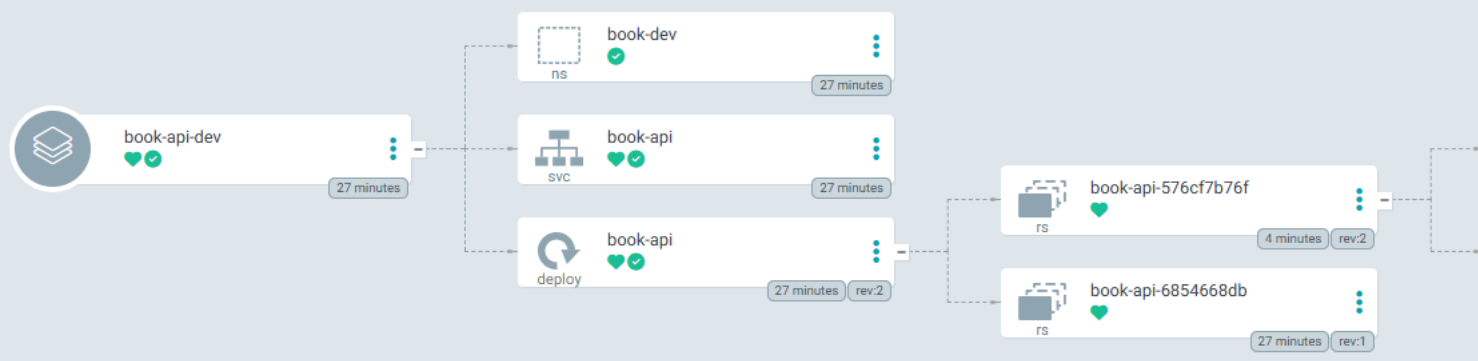
LAST SYNC 🟢 **Sync OK** to af9cb53 [🔗](#)

Succeeded 4 minutes ago (Fri Mar 13 2026 13:00:08 GMT+0900)

Author: leeyoungkon <yklee2002@gmail.com> -

Comment: libe probe 지움

☰ ☰ ☰ + - 🔍 100%



```

graph LR
    book-api-dev[book-api-dev ns] --- book-api-svc[book-api svc]
    book-api-dev --- book-api-deploy[book-api deploy]
    book-api-svc --- book-api-576cf7b76f[book-api-576cf7b76f rs]
    book-api-svc --- book-api-6854668db[book-api-6854668db rs]
  
```

커맨드 창에서도 확인

```
C:\Users\leeyoungkon>kubectl get all -n book-dev
```

NAME	READY	STATUS	RESTARTS	AGE
pod/book-api-576cf7b76f-kk5m7	1/1	Running	0	10s
pod/book-api-576cf7b76f-qzq8p	1/1	Running	0	21s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/book-api	ClusterIP	10.110.16.218	<none>	80/TCP	23m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/book-api	2/2	2	2	23m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/book-api-576cf7b76f	2	2	2	21s
replicaset.apps/book-api-6854668db	0	0	0	23m

ArgoCD에서 자동 sync 설정: details 클릭

← → ↺ 🏠 주의 요함 <https://localhost:8080/applications/argocd/book-api-dev?view=tree&resource=>

📦 새 탭 8 Container Orches... 100 Committee ISO/TC... 100 ISO/TC 307 - Block... <📁> 쿠버네티스 시작하... 🌐 클라우드스토어 써... 🏠 Home Page - Clou... 📊 [시장동향] 대세는... 📈 큰 숲 맑은 샘 🌐 대한성서

argo v3.3.3

Applications Settings User Info Documentation

Resource filters

NAME NAME

KINDS KINDS

SYNC STATUS

☐ Synced 3

☐ OutOfSync 0

HEALTH STATUS

☐ Progressing 0

Applications / Q book-api-dev

DETAILS DIFF SYNC SYNC STATUS HISTORY AND ROLLBACK DELETE REFRESH

APP HEALTH **Healthy**

SYNC STATUS **Synced** to HEAD (af9cb53) [🔗](#)

Auto sync is not enabled.

Author: leeyoungkon <ykle2002@gmail.com> - Comment: libe probe 지움

LAST SYNC **Sync OK** to af9cb53 [🔗](#)

Succeeded 4 minutes ago (Fri Mar 13 2026 13:00:08 GMT+0900)

Author: leeyoungkon <ykle2002@gmail.com> - Comment: libe probe 지움

☰ ☰ ☰ + - 🔍 100%

book-api-dev 27 minutes

book-dev ns 27 minutes

book-api svc 27 minutes

book-api deploy 27 minutes rev:2

book-api-576cf7b76f rs 4 minutes rev:2

book-api-6854668db rs 27 minutes rev:1

ArgoCD에서 자동 sync 설정: summary tab의 auto sync 체크

The screenshot displays the ArgoCD application summary page. The left sidebar shows the 'DETAILS' tab selected, with 'APP HEALTH' indicating 'Health' with a green heart icon. The main content area is divided into two sections. The top section, 'Summary', lists application details: TARGET REVISION (HEAD), PATH (overlays/dev), SYNC OPTIONS, RETRY OPTIONS (Retry disabled), STATUS (Synced to HEAD (af9cb53)), HEALTH (Healthy), LINKS, and IMAGES (ykee2002/book-api:latest). The bottom section, 'SYNC POLICY', contains the 'AUTOMATED' settings. The 'ENABLE AUTO-SYNC' checkbox is checked and circled in red. Below it are unchecked checkboxes for 'PRUNE RESOURCES' and 'SELF HEAL'.

Summary	
TARGET REVISION	HEAD
PATH	overlays/dev
SYNC OPTIONS	
RETRY OPTIONS	Retry disabled
STATUS	Synced to HEAD (af9cb53)
HEALTH	Healthy
LINKS	
IMAGES	ykee2002/book-api:latest

SYNC POLICY	
AUTOMATED	
☒ ENABLE AUTO-SYNC	
☐ PRUNE RESOURCES	
☐ SELF HEAL	

도커 이미지 레지스트리

✓ 로그인 url: <http://210.90.24.172:8080>, id : admin, pwd : Harbor12345

The screenshot displays the Harbor web interface on a browser at the URL `210.90.24.172:8080/harbor/projects`. The interface shows a sidebar with navigation options like '프로젝트' (Projects), '로그' (Logs), '관리' (Management), and '설정' (Settings). The main area is titled '프로젝트' and shows a table of projects.

Overlaid on the right is a terminal window showing the command `sudo docker compose up -d` being executed. The output indicates that all services are running successfully:

```
ubuntu@ubuntu:~/harbor$ sudo docker compose up -d
[+] Running 10/10
✓Network harbor_harbor      Created
✓Container harbor-log       Started
✓Container harbor-db        Started
✓Container registry         Started
✓Container registryctl      Started
✓Container redis            Started
✓Container harbor-portal    Started
✓Container harbor-core      Started
✓Container harbor-jobservice Started
✓Container nginx            Started
```

Below the terminal, the Harbor web interface shows a table of projects:

프로젝트 이름	엑세스 레벨	역할	종류	저장소 수	생성 시간
library	공개	프로젝트 관리자	프로젝트	0	26. 5. 1. PM 11:25

At the bottom right of the table, it indicates '페이지 크기 15' and '1 - 1 of 1 아이템'.

도커 이미지 레지스트리

- ✓ 프로젝트 생성 후 사용자 등록하고, 그 사용자를 프로젝트에 등록해야 함
- ✓ 프로젝트 레포지토리는 이미지를 처음 push 할 때 자동 생성

☐	이름	관리자 ⓘ
☐	leeyoungkon	아니요
☐	ykleee2002	아니요

CI.yaml (Harbor 사용에 따른 변경 부분)

```
env:  
  HARBOR_REGISTRY: 210.90.24.172:8080  
  HARBOR_PROJECT: restproject  
  STUDENT_ID: leeyoungkon  
  IMAGE_NAME: resttemplateserver
```

repository name이 됨

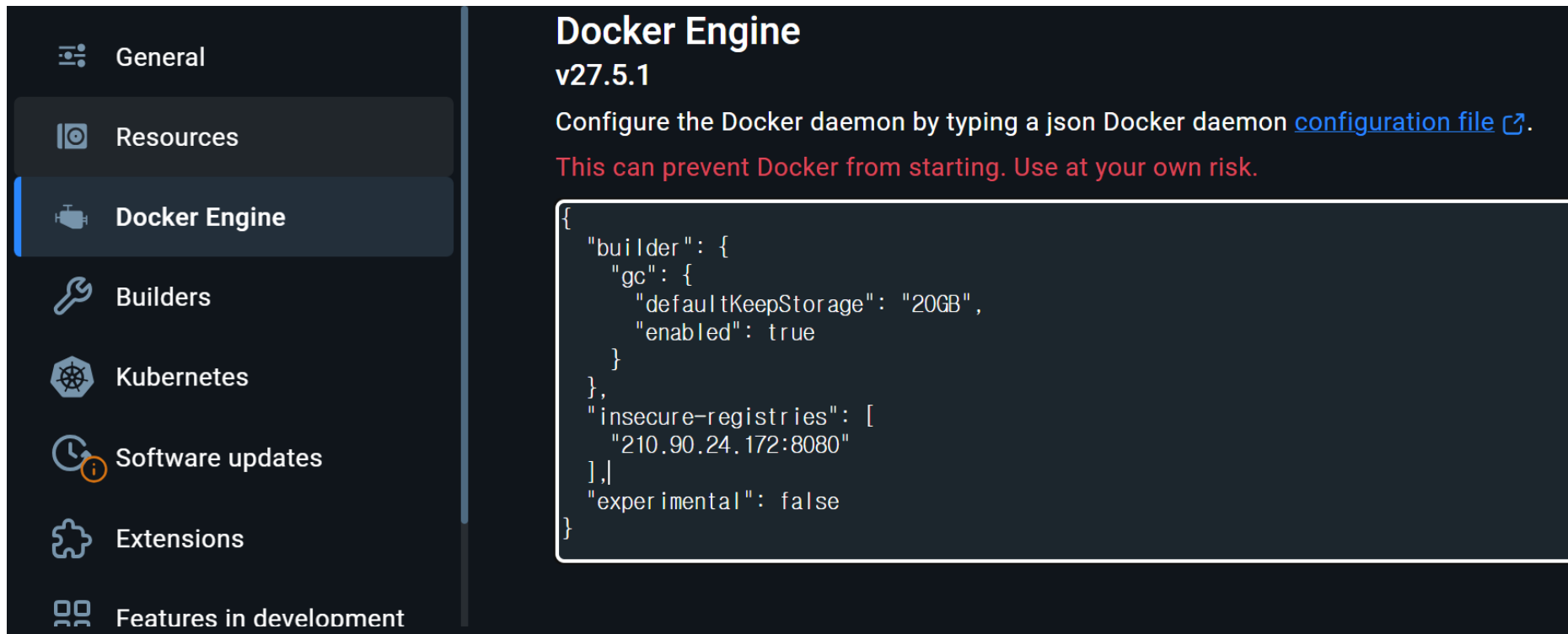
```
registry: ${ env.HARBOR_REGISTRY }  
username: ${ secrets.HARBOR_USERNAME }  
password: ${ secrets.HARBOR_PASSWORD }  
  
name: Build and push Docker image  
uses: docker/build-push-action@v5  
with:  
  context: .  
  push: true  
  tags: |  
    ${ env.HARBOR_REGISTRY }/${ env.HARBOR_PROJECT }/${ env.STUDENT_ID }/${  
    ${ env.HARBOR_REGISTRY }/${ env.HARBOR_PROJECT }/${ env.STUDENT_ID }/${
```

k8s.yaml 변경 부분

```
spec:
  containers:
  - name: resttemplateserver
    image: 210.90.24.172:8080/restproject/leeyoungkon/resttemplateserver:eddda1e8bf5d9538f
    imagePullPolicy: Always
    ports:
    - containerPort: 8080
```

[Harbor registry ip] [프로젝트명] [레포지토리명] [이미지명: SHA 태그]

도커 엔진에 insecure registry 정보 추가



Docker Engine
v27.5.1

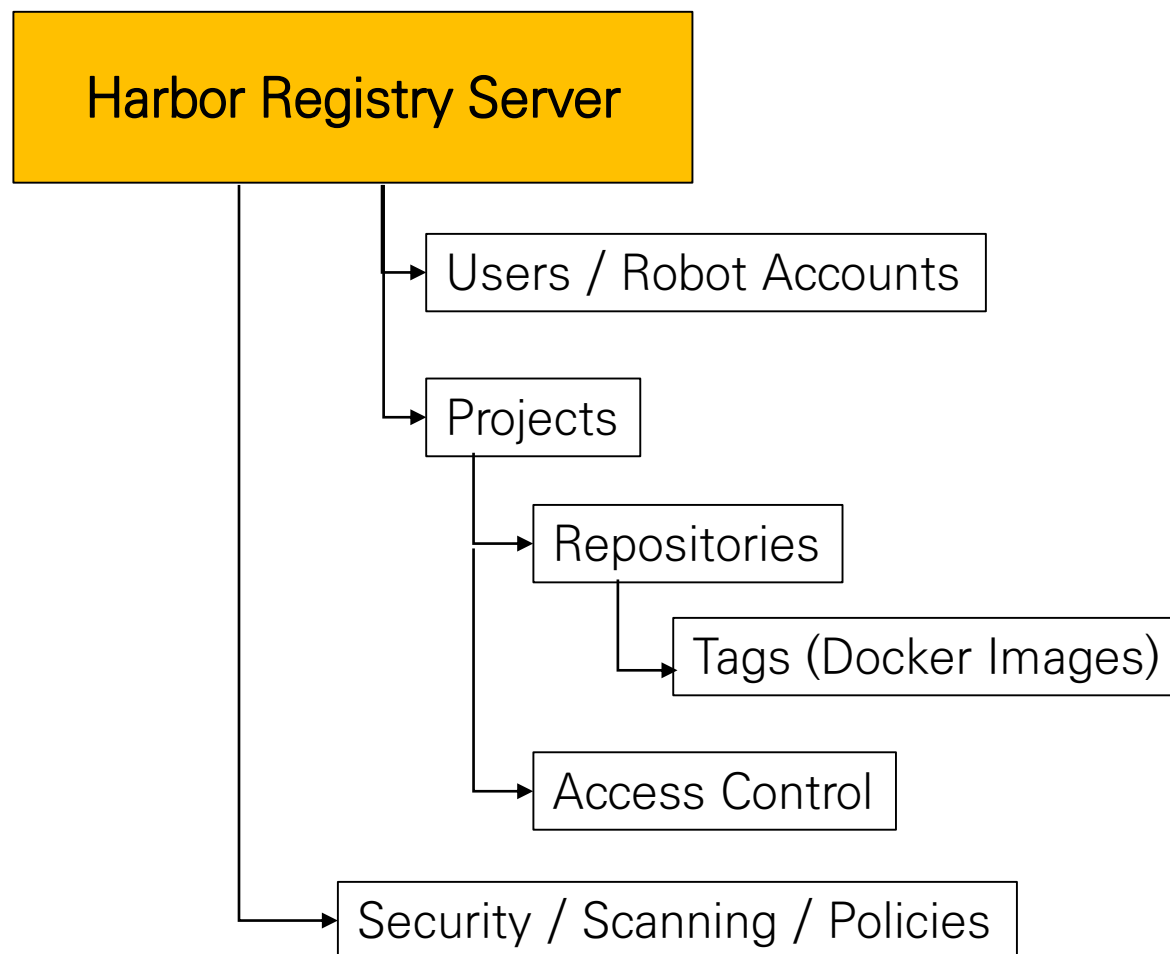
Configure the Docker daemon by typing a json Docker daemon [configuration file](#).
This can prevent Docker from starting. Use at your own risk.

```
{
  "builder": {
    "gc": {
      "defaultKeepStorage": "20GB",
      "enabled": true
    }
  },
  "insecure-registries": [
    "210.90.24.172:8080"
  ],
  "experimental": false
}
```

Harbor를 insecure 레지스트리로
등록하고, http로 접근하게 하기 위함

```
"insecure-registries": [
  "210.90.24.172:8080"
]
```

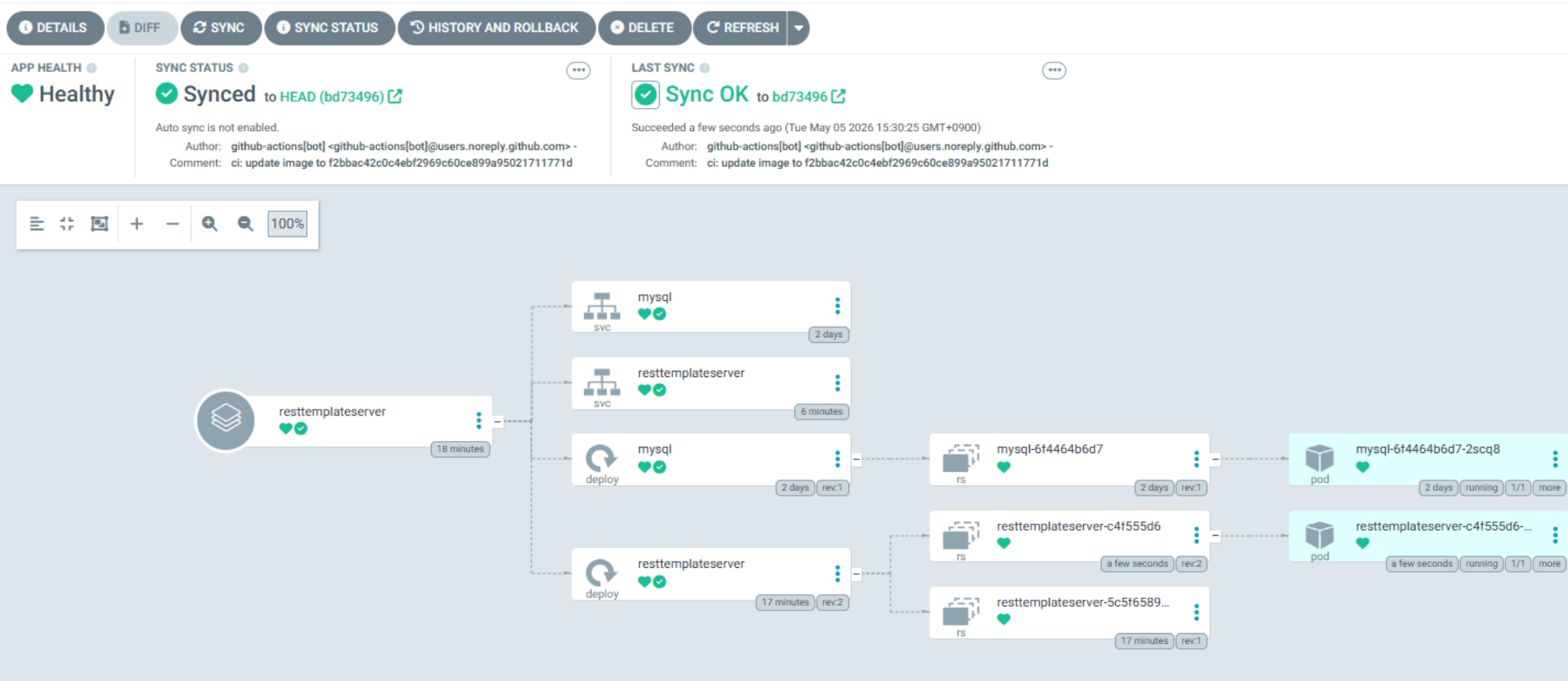
Harbor 구성



Harbor 요소 설명

요소	필요 여부	설명
Harbor 서버(Registry)	필수	전체 저장소
Project	필수	push 대상
User/Robot Account	필수	인증
Project 권한	필수	push 허용
GitHub Secrets	필수	CD 인증
Repository	자동 생성	첫 push 시

CI/CD 현황 보기



감사합니다.